

# ■ 77-DAY WAR PLAN

Devaraj S • IIITDM Kancheepuram • Summer 2026

C++ Mastery → Vortex GPGPU Critical Contributions → International Intern

77  
DAYS

2 HRS  
/ DAY

2  
PHASES

1  
RESOURCE C++

∞  
VORTEX IMPACT

## PHILOSOPHY

You are not learning C++ to pass interviews. You are learning C++ because **GPU architects live in C++**. Every CUDA kernel, every RTL sim driver, every Vortex pipeline is C++. One resource. No rabbit holes. Execute.

## THE ONE C++ RESOURCE

**learncpp.com**

Why only this?

Free · Browser · No fluff · GPU-relevant depth

Covers exactly what you need: pointers, memory model, templates, move semantics, bitwise ops, STL — all things Vortex contributors and GPU architects use daily. It is not a bloated textbook. It teaches the way engineers think.

## PHASE 1 — C++ FOUNDATIONS + DSA

Days 1 – 35 • 1 hr C++ (learncpp.com) + 1 hr DSA (LeetCode)

### WEEK 1 (Days 1–7)

#### C++ (1 hr) — Fundamentals & Memory Model

- Ch 1–4 learncpp: variables, types, literals, I/O
- Ch 5: operators, precedence
- Ch 9–10: type conversions, references
- Ch 11: function basics, overloading

*GPU relevance: Stack vs heap, value vs reference — critical for understanding GPU memory spaces (global/shared/register)*

#### DSA (1 hr) — Arrays, basic iteration, hash maps

- Two Sum (Easy) — LC 1
- Valid Parentheses (Easy) — LC 20
- Best Time to Buy/Sell Stock (Easy) — LC 121
- Contains Duplicate (Easy) — LC 217
- Maximum Subarray (Med) — LC 53

### WEEK 2 (Days 8–14)

#### C++ (1 hr) — Pointers, Arrays & Raw Memory

- Ch 12: pointers, null pointers, pointer arithmetic
- Ch 17: arrays, C-style arrays, std::array
- Ch 13: enums, structs
- Manual memory: new / delete patterns

*GPU relevance: Pointer arithmetic is how you navigate GPU memory buffers. Every Vortex memory subsystem is pointer math.*

#### DSA (1 hr) — Pointer-style thinking in arrays, two-pointer

- Merge Sorted Array (Easy) — LC 88
- 3Sum (Med) — LC 15
- Product of Array Except Self (Med) — LC 238
- Find Minimum in Rotated Sorted Array (Med) — LC 153
- Median of Two Sorted Arrays (Hard) — LC 4

### WEEK 3 (Days 15–21)

#### C++ (1 hr) — OOP + Templates Intro

- Ch 14–15: classes, constructors, destructors
- Ch 14: access specifiers, member functions
- Ch 16: operator overloading
- Ch 26: templates (function & class templates)

*GPU relevance: Vortex codebase uses templated pipeline stages. CUDA abstractions are class-based. Operator overloading used in sim frameworks.*

#### DSA (1 hr) — STL containers, design problems

- LRU Cache (Med) — LC 146
- Implement Stack using Queues (Easy) — LC 232
- Min Stack (Med) — LC 155
- Design HashMap (Easy) — LC 706
- Binary Search (Easy) — LC 704

### WEEK 4 (Days 22–28)

### C++ (1 hr) — Move Semantics, Smart Pointers, RAII

- Ch 22: move semantics, rvalue references
- Ch 22: unique\_ptr, shared\_ptr, weak\_ptr
- RAII pattern — resource management
- Ch 18: inheritance, virtual functions

*GPU relevance: Vortex sim allocates/frees large memory structs constantly. Smart pointers prevent leaks. RAII is how sim resources are managed.*

### DSA (1 hr) — Linked list = manual pointer + RAII thinking

- Linked List Cycle (Easy) — LC 141
- Merge Two Sorted Lists (Easy) — LC 21
- Reverse Linked List (Easy) — LC 206
- Remove Nth Node From End (Med) — LC 19
- Copy List with Random Pointer (Med) — LC 138

## WEEK 5 (Days 29–35)

### C++ (1 hr) — Bit Manipulation, Concurrency Intro, STL Deep

- Bitwise: AND, OR, XOR, shifts, masks
- Bit fields in structs (used in ISA decoders!)
- Ch 23: STL algorithms (sort, find, accumulate)
- Ch 27: intro to threads (std::thread basics)

*GPU relevance: ISA decoding in Vortex is 100% bit manipulation. Warp IDs, masks, lane enables — all bitwise. Threads for parallel simulation.*

### DSA (1 hr) — Bit manipulation — directly GPU-ISA relevant

- Number of 1 Bits (Easy) — LC 191
- Reverse Bits (Easy) — LC 190
- Single Number (Easy) — LC 136
- Counting Bits (Easy) — LC 338
- Sum of Two Integers (Med) — LC 371

## ■ PHASE 1 CHECKPOINT (Day 35)

You know: pointers · templates · move semantics · bit ops · STL · RAII You've solved: 25 curated LeetCode problems Resume line: C++ (learncpp.com) — pointers, templates, bit manipulation, STL, RAII | 25 LeetCode (Easy–Hard)

# PHASE 2 — VORTEX GPGPU CRITICAL CONTRIBUTIONS + DSA

Days 36 – 77 • 1 hr Vortex + 1 hr DSA (LeetCode)

## VORTEX GPGPU — WHAT YOU MUST KNOW FIRST

|           |                                                                                      |
|-----------|--------------------------------------------------------------------------------------|
| REPO      | github.com/vortexgpgpu/vortex — RISC-V based GPGPU, open-source, academic-grade      |
| LANGUAGE  | C++ (sim/driver) + SystemVerilog (RTL) — your C++ from Phase 1 maps directly         |
| KEY DIRS  | hw/ → RTL   sim/ → C++ simulator   runtime/ → GPU driver   tests/ → benchmarks       |
| BUILD     | cmake + make — read CMakeLists.txt first, understand the build graph                 |
| SIM ENTRY | sim/simx/ → main C++ simulator — start here, trace execution flow                    |
| PIPELINE  | Fetch → Decode → Issue → Execute → Writeback — standard GPU pipeline in RTL+sim      |
| WARP SIM  | sim/simx/warp.cpp — warp state machine — this is where scheduling lives              |
| ISA       | RISC-V RV32I + custom GPU extensions — your RV32I Verilog project is directly useful |

### WEEK 6 (Days 36–42) — REPO MASTERY

#### Vortex (1 hr)

*Clone, build, run. Read sim/simx/ entirely. Trace a single warp through the pipeline. Draw the execution flow yourself.*

- Clone + build with all targets (simx, rtlsim)
- Run all tests/benchmarks — record baseline numbers
- Read warp.cpp + core.cpp + pipeline stages
- Annotate: where are scheduling decisions made?
- Open issues tab — categorize open issues by type

#### DSA (1 hr) — Graph — models GPU execution graphs

- Number of Islands (Med) — LC 200
- Clone Graph (Med) — LC 133
- Course Schedule (Med) — LC 207
- Pacific Atlantic Water Flow (Med) — LC 417
- Word Ladder (Hard) — LC 127

### WEEK 7 (Days 43–49) — WARP SCHEDULER DEEP DIVE

#### Vortex (1 hr)

*Understand current warp scheduling policy (round-robin). Profile which warps stall. Identify divergence patterns.*

- Instrument scheduler — add stall counters per warp
- Run SAXPY + SGEMM — capture warp utilization traces
- Read GPU scheduling papers (GTO, two-level, CCWS)
- Identify: does Vortex have divergence-aware scheduling? (No — opportunity!)
- Write issue/proposal: 'Divergence-Aware Warp Scheduling'

#### DSA (1 hr) — Heaps + scheduling — maps to warp scheduler priority queue

- Top K Frequent Elements (Med) — LC 347
- Find Median from Data Stream (Hard) — LC 295
- Kth Largest in Array (Med) — LC 215
- Task Scheduler (Med) — LC 621
- Reorganize String (Med) — LC 767

### WEEK 8 (Days 50–56) — FIRST CRITICAL CONTRIBUTION

### Vortex (1 hr)

*Implement + PR: Enhanced warp scheduler with stall-detection heuristic OR performance counters framework.*

- Pick target: warp scheduler improvement OR perf counters
- Create branch: feature/divergence-aware-scheduler
- Implement: priority queue based scheduler with stall tracking
- Add: per-warp utilization metric to sim output
- Test: compare SAXPY/SGEMM numbers before/after

### DSA (1 hr) — Monotonic stack — models instruction windows

- Trapping Rain Water (Hard) — LC 42
- Largest Rectangle in Histogram (Hard) — LC 84
- Daily Temperatures (Med) — LC 739
- Next Greater Element I (Easy) — LC 496
- Sliding Window Maximum (Hard) — LC 239

## WEEK 9 (Days 57–63) — MEMORY SUBSYSTEM + SECOND PR

### Vortex (1 hr)

*Study Vortex cache/memory hierarchy. Identify bottleneck. Contribute: cache miss tracking, prefetch hint, or coalescing improvement.*

- Read sim/simx/cache\_sim.cpp thoroughly
- Profile memory access patterns in SGEMM benchmark
- Identify: coalescing inefficiency or bank conflict
- Implement: memory access coalescing checker / visualizer
- Open PR with benchmark data proving the improvement

### DSA (1 hr) — Cache + memory design problems

- LRU Cache (Med) — LC 146 (revisit — implement in C++)
- All O'one Data Structure (Hard) — LC 432
- Insert Delete GetRandom O(1) (Med) — LC 380
- Design In-Memory File System (Hard) — LC 588
- Design Search Autocomplete (Hard) — LC 642

## WEEK 10 (Days 64–70) — BENCHMARKING + DOCUMENTATION PR

### Vortex (1 hr)

*Add missing benchmarks or fix documentation gaps. These are highly valued by maintainers — low friction, high visibility.*

- Identify missing benchmark (sparse matmul? attention?)
- Implement + validate new benchmark
- Profile with new scheduler — show speedup table
- Fix any build warnings / CI issues you spotted
- Write detailed README additions for your contributed features

### DSA (1 hr) — Trees + advanced — models pipeline trees

- Serialize and Deserialize Binary Tree (Hard) — LC 297
- Binary Tree Maximum Path Sum (Hard) — LC 124
- Lowest Common Ancestor (Med) — LC 236
- Recover Binary Search Tree (Med) — LC 99
- Count of Smaller Numbers After Self (Hard) — LC 315

## WEEK 11–11.5 (Days 71–77) — OUTREACH + WRAP

### Vortex (1 hr)

*Email the Vortex maintainers. Write a 1-page research note on your scheduler contribution. Prepare everything for resume.*

- Send professional email to Prof. Tine (Vortex PI) — subject: 'Contributions + Interest in Research Collaboration'
- Attach: link to your merged PRs, benchmark results
- Write: 1-page 'Scheduling Improvements in Vortex GPGPU' — post to your GitHub
- Update resume: C++ skills + Vortex contributions + LeetCode count
- Polish all open PRs — respond to every review comment promptly

### DSA (1 hr) — DP — shows algorithmic depth for grad school

- Word Break II (Hard) — LC 140
- Edit Distance (Hard) — LC 72
- Longest Increasing Subsequence (Med) — LC 300
- Burst Balloons (Hard) — LC 312
- Regular Expression Matching (Hard) — LC 10

FINAL DELIVERABLES — DAY 77

|               |                                                                                                             |
|---------------|-------------------------------------------------------------------------------------------------------------|
| RESUME LINE 1 | C++: pointers, templates, move semantics, smart pointers, bit manipulation, STL, concurrency (learncpp.com) |
| RESUME LINE 2 | LeetCode: 35 problems solved — arrays, graphs, heaps, DP, bit manipulation (Easy to Hard)                   |
| RESUME LINE 3 | Open-source: 2–3 merged PRs to Vortex GPGPU (warp scheduler + memory + benchmarks)                          |
| RESUME LINE 4 | Research note: 'Scheduling Improvements in Vortex GPGPU' — available on GitHub                              |
| NETWORK       | Direct email sent to Vortex PI — known contributor, not anonymous applicant                                 |
| INTERN TARGET | Any international GPU/architecture lab now has hard evidence of your depth                                  |

THE EMAIL THAT GETS YOU NOTICED (send Day 77)

Subject: Contributions to Vortex GPGPU + Interest in Research Collaboration Prof. Tine / Vortex Team, I am Devaraj S, a 2nd-year undergrad at IIITDM Kancheepuram specializing in computer architecture and AI accelerators. Over the past 6 weeks I contributed to the Vortex GPGPU repository: • PR #XXX – Divergence-aware warp scheduler with stall-detection heuristic (X% IPC improvement on SGEMM) • PR #XXX – Memory access coalescing analysis framework • PR #XXX – Sparse matrix benchmark suite I am currently interning at IIT Madras on neuromorphic accelerator scheduling (APEX accelerator group). My long-term goal is GPU architecture research at the MS level (Stanford/CMU/Berkeley). I would be very grateful for any guidance on how I can contribute more meaningfully to Vortex, or if there are any open research problems in the project where I could help. GitHub: github.com/[your-handle] | Research note: [link] Thank you for building Vortex – it is the best open-source GPGPU platform for learning architecture. Devaraj S

DAILY RHYTHM — NON-NEGOTIABLE

| SLOT        | PHASE 1 (Days 1–35)                                  | PHASE 2 (Days 36–77)                                 |
|-------------|------------------------------------------------------|------------------------------------------------------|
| Hour 1      | learncpp.com — read + code every example             | Vortex codebase — one task from the week list        |
| Hour 2      | 1 LeetCode problem — solve, then read best solution  | 1 LeetCode problem — solve, then read best solution  |
| Daily rule  | Commit every code snippet to GitHub (shows activity) | Every Vortex change → branch → commit → eventually P |
| Weekly rule | 1 paragraph note on what you learned this week       | Update your research note with new findings          |

2 hours a day. Every day. No breaks. By Day 77 you will have shipped real code to a real GPU simulator that researchers worldwide use. That is not a resume line — that is a proof of work that no LOR can fake.